PRIVATE · PREPARED FOR BHAVIT

Under the Hood

The answers behind the answers — so that no follow-up question can knock you over.

COMPANION TO	The NEA Meeting — 53 pages, 24 sections
MEETING	Thomas Joshi, Associate — NEA Friday 23 October 2026 · 11:00am Pacific
COVERS	The engine, the evaluation, the learning boundary, the defensibility argument
RULE	Where the code contradicts the deck, the code wins

Prepared for Bhavit Saini. Private and confidential. Every factual claim carries an A–D grade. Where something is not built, this book says so.

Contents

PART I — PLAINLY

00 What this book is 3
01 How Evo works, in plain English 5
02 The four seconds after you say “continue” 7

PART II — THE MACHINE

03 The evaluation harness, file by file 10
04 Does it learn? 12
05 Where does it learn if nothing leaves the device? 15
06 Measuring time-to-resume 17
07 Four jobs, fifteen windows 19
08 The four things it cannot do 21

PART III — DEFENCE

09 Privacy is the licence, not the moat 22
10 The category is bigger than memory 23
11 Who else is building this 24
12 How the layer actually gets built 25
13 “Is it big enough?” 26

PART IV — THE ROOM

14 The four answers, rewritten 27
15 The demo when the model isn't learning 29
16 Incorporation, visas, and the $83,000 30
17 When you don't know 32
18 The five sentences 34

If you have one hour: read 01, then 03, then 07, then 09. Those four cover the plain explanation, the evaluation, the hardest technical question, and the defensibility argument — which between them are most of what Thomas will actually probe.

READ ME FIRST

00 What this book is

This is the companion to The NEA Meeting . That book tells you what to say on Friday. This one explains **why the answers are true** — the machinery underneath, in enough depth that no follow-up question can knock you over.

It exists because of a specific problem. The main book gives you sentences. Sentences are fragile: the moment Thomas asks "how?" a memorised line turns into a bluff. What makes a sentence survive a follow-up is knowing the thing behind it. That is what these pages are for.

How it's organised

Part	What it covers	When to read it
I. Plainly	How Evo works with no jargon; the exact sequence of events when you say "continue."	Tonight. It is the foundation for everything else.
II. The machine	The evaluation harness file by file; whether it learns and how it would; where learning happens without a cloud; how time-to-resume is measured; how it separates four jobs across fifteen windows.	Tonight or tomorrow morning. This is where confidence comes from.
III. Defence	The moat argument, why fleet priors are not a moat, who else is building this, how the layer gets built, and the answer to "is it big enough."	Tomorrow. Read before the ones that need it, not before Friday.
IV. The room	Four technical answers rewritten from earlier drafts; the demo when the model isn't learning; incorporation and visas; what to say when you don't know.	Thursday night and Friday morning.

The one rule that governs every page

> **WHY THIS WORKS**
>
> Where the code contradicts the deck, the code wins. Not because honesty is virtuous — because Thomas can read code, and a claim that survives the meeting but dies in diligence is worse than one that dies on Friday. Every time you correct yourself before he does, you buy credibility for everything you don't correct.

Grades

Every factual claim carries one of four marks:

- **A** — verifiable: the repository, a primary document, or a paper you can name.
- **B** — reputable secondary source, or a defensible inference. Fine to say, with a hedge.
- **C** — directional. Useful for framing, not for quoting as fact.
- **D** — do not repeat. Either wrong, or unverifiable in a way that would embarrass you.

And here is the grade attached to every load-bearing fact in this book, so that you can see the difference between what you know and what you have been told. If he asks "how do you know that", the answer is in the third column.

Fact	Grade	Basis
The engine has no HTTP client	A	crates/ tree; a grep for a network call returns nothing.
The labelled corpus is six works from one day, 164 labels	A	sample_events.json, ground_truth_labels.json.
The gate hard-fails on fusion and on a leisure anchor	A	Six assertions in bin/regression.rs.
The restore cap is five resources	A	restore.rs, module docs.
1,272 tests pass across 59 suites	A	BUILD-LOG.md, 30 Sep; the release receipt in §23 of the main book.
NEA manages more than $38B	A	NEA’s own published figure, June 2026.
NEA 19 is roughly a $5.5B fund	B	Reported at first close, Dec 2025–Jan 2026.
Thomas co-authored DSPy and owns evaluation at NEA	A	His NEA bio and his own writing.
Median post-money SAFE cap is $18M for $1M–$2.4M rounds	B	Carta Q2 2026, via two secondary write-ups.
OpenAI’s Computer History is macOS-only and stores nothing but events	B	Launch docs, 13 Aug 2026.
NEA likes "newly captured workflow data"	C	Their own thesis language; the phrase is theirs, the inference is mine.

Two of those being **C** is not a weakness in the document. It is the point: the difference between the two columns is the difference between what you can defend under questioning and what you should say once, hedged, and move on from.

How to read it

Four parts, and they are ordered by how much you already know rather than by importance.

Part	What it does	Read it if
I — Plainly	Explains the product, the engine, and the four seconds with no jargon at all.	You have to explain Evo to someone who has never heard of it — including yourself, at 11pm.
II — The machine	The evaluation harness, the learning boundary, the metric, and the limits.	He asks a technical question and you want the answer to be true rather than confident.
III — Defence	Privacy, category, competitors, the build plan, and the size question.	He pushes back, which he will, because that is his job.
IV — The room	Your four answers rewritten, the demo, money and visas, and what to say when you don’t know.	It is the night before, or you are in the car.

You do not have to read it in order, and you should not try to memorise it. The sections that matter most for a thirty-minute associate call are **01** , **03** , **07** and **09** — the explanation, the evidence, the hardest technical claim, and the answer to "why won’t Apple do this?"

Everything below is written to be understood on one pass. If a paragraph requires you to hold three clauses in your head to follow it, that is a fault in the writing, not in you — and it is the exact fault that loses a meeting, because you cannot reproduce a sentence you had to decode.

NO JARGON

01 How Evo works, in plain English

The one sentence

> **SAY THIS**
> Evo watches how you work, groups it into your separate projects, and puts your desk back exactly as it was when you come back.

That is the whole product. Everything else in this book is detail underneath those three clauses.

The desk

Picture your computer as a desk. All day you pile things on it — documents, browser tabs, files, folders, terminal windows. You're working on four different things at once and it's a mess, but it's your mess and you know where everything is.

Then the phone rings. You take a forty-minute call. Or you go to lunch. Or you sleep.

You come back and you've lost it. Not the files — they're all still there. You've lost **where you were** . So you spend ten minutes clicking tabs, opening folders, trying to remember which of the forty open windows was the one that mattered.

Evo is the assistant who was sitting next to you the whole time. When you come back, everything else is cleared away and the right pile is in front of you — the document on top, the three tabs you needed, the folder open underneath. You don't search, you don't remember. You just keep going.

How it knows

Three mechanisms, in order. This is the part worth internalising, because it is the answer to almost every technical question he can ask.

It watches

A small program runs in the background on your Mac. It notices which window you're looking at, what file you saved, what page you opened, what you typed into, when you committed code. It does this all day, quietly.

It never takes screenshots. It isn't recording your screen. It sees what you're working in and when — not what's inside it. That distinction is the entire privacy argument, and it is also the reason the product can run for weeks without eating your battery.

It groups

Everything you do gets chopped into chunks: a stretch of time where you stayed on one thing. Type in a file, save it, switch to the browser, come back, save again — that's one chunk of one job.

The cutting is done by content, not by the clock. If you glance at a notification for ninety seconds and come back, that's not two chunks — it's one chunk with an interruption in it. Only a gap long enough to be a real departure (twenty minutes of silence, by default) ends the chunk.

It decides what belongs to what

This is the actual invention, and it is one idea:

> **SAY THIS**
> It doesn't look at what you have open. It looks at what you produced.

Things you only looked at are context. Things you typed in, saved, downloaded, or committed to are identity. That single rule is why a browser with fifteen tabs across three different projects doesn't confuse it — the tabs you didn't work in don't get a vote.

And if something genuinely belongs to two projects, **it refuses to guess.** It shows you both and lets you pick. It would rather be unsure than be confidently wrong.

It remembers

When you correct it — drag a window into the right project, merge two, split one — that correction is permanent and outranks anything it figured out on its own. You teach it once.

What you do

You say: "Hey Evo, continue the pitch deck."

It figures out which project you mean, then reopens what that project needs — the document, the tabs you had open alongside it, the folder. Five things at most. Not everything you ever touched, just enough to pick up where you left off.

No searching. No file browser. No remembering. You were interrupted; now you're back.

Why it's different from everything else

Every other tool answers **"where is that thing?"** — Spotlight, Recall, Rewind, ChatGPT's memory, your browser history. They are all search. They all assume you know what you're looking for and just can't find it.

Evo answers a different question: **"where was I?"**

That is not a search problem. It is a state problem — what was open, in what arrangement, on this machine. Nothing on the web can answer it, because the answer isn't on the web. It's on your laptop, in the mess you left behind.

> **SAY THIS**
> "Search assumes you know what you lost. Evo knows where you were."

> **WHY THIS WORKS**
> That line is worth memorising. It is the whole positioning in twelve words, it is true, and it survives any follow-up because it is a statement about the shape of the problem rather than a claim about your technology.

THE SEQUENCE

02 The four seconds after you say "continue"

If he asks "walk me through what happens," this is the answer. Six stages, roughly four seconds end to end, all of it on the machine. You do not need to recite all six — you need to know them so that a question about any one of them doesn't strand you.

#	What happens	Where it lives	Roughly
1	Wake word detected acoustically, on-device. Microphone buffers are processed locally by a small ONNX model.	evo-voice	instant
2	Whisper transcribes the bounded utterance. Local model file, no network.	evo-voice	~1 s
3	The transcript is embedded and matched against your works by meaning, not keywords. A winner must clearly beat the runner-up, or the answer is "ambiguous" rather than a guess.	evo-semantic, evo-retrieval	milliseconds
4	The selected work yields a restore set: a resume point first, then the supporting surfaces, capped at five.	evo-threads	milliseconds
5	Each resource becomes an action. A URL opens in the browser it was seen in. A file opens in its owning app — or, if a live window already shows that exact document, that window is raised instead.	evo-execution	0.2–3 s
6	Every target is reported into one of three buckets — opened, raised, or unopened-with-reason — and each attempt carries a verification result: Verified, Unverified, or Failed.	evo-execution	instant

Why stage five is the interesting one

Anybody can launch an app. The hard part is that **restoring is not opening** . If the document is already open in a window somewhere, opening it again gives you two copies and a worse desk than you started with. So the executor checks first, and raises rather than duplicates.

That is also why there's a whole module about refusing to act. If two windows have the same title and the engine can't tell which one you meant, it does not pick one. It refuses and says so. Early versions of this code picked the first match — the repo's own audit found it, and it was fixed, because picking the first match is how you open a client's document in the wrong client's window.

Why stage six is the most important one

This is the part almost no consumer product does, and it is the thing to point at when he asks how you know it works.

A restore that silently skipped half the neighbourhood is worse than one that says "I couldn't open this." The person trusts the second one and corrects it. The first one teaches them the product lies, and they stop using it within a week.

SAY THIS

"Every restore reports what it opened, what it brought forward, and what it couldn't open — three buckets, no silent skips. And each attempt carries a verification result, because the operating system accepting a command is not proof that the document appeared."

WHERE THIS COMES FROM

crates/evo-execution/src/restore.rs, module documentation and VerificationResult. Read the doc comment at the top of the file — it explains the design in better words than this book does.

If he asks how long a restore actually takes

Be careful here, because this is a number you have not measured on his hardware. Give the shape rather than a figure:

SAY THIS

"It depends almost entirely on the applications, not on the engine. Finding the work is milliseconds — it's a lookup, not a model call. Opening things is the variable: a file and a URL are fast, a cold application launch is a few seconds, and an app that has to reconnect to something is slower than I control. What I'd want to publish is the number that matters — seconds from 'continue' to first productive action — and I'd rather measure it across real users than quote you my own machine."

WHY THIS WORKS

That answer does three things: it separates the fast part from the slow part, which shows you understand where the cost lives; it names the metric that actually matters; and it refuses to invent a number without sounding evasive. Associates hear "a few seconds" as marketing and "it depends on the applications, here's why" as engineering.

What the restore set contains

The cap is five resources. The composition, in order of priority:

Slot	What goes in it	Why
1. The resume point	The single resource you were producing in when the episode ended.	This is the one window that makes the difference between "back" and "nearly back."
2–4. Supporting surfaces	Other production-bearing resources in the same work, in evidence order.	The tabs and files you were actually working across.
5. Reference companion	At most one attention-only resource, if its weight clears the floor.	The document you were reading from. It decorates; it cannot lead.
Never	Leisure, comms, or anything the person classified out.	Those are context at best and noise at worst. The gate hard-fails if one ever anchors a work.

Five is a product decision, not a technical limit. Restoration reopens the neighbourhood, not the museum. A restore set of twelve windows is a worse experience than five, because you have to look at all twelve to find the one that mattered — which is the exact problem the product exists to remove.

What the four seconds are not

> **CAREFUL**
> It is not a chat. There is no model generating text, no prompt, no response. The voice path is a command channel, not a conversation. If he asks "what model do you use for the assistant," the honest answer is: "There isn't one in the loop. There's a wake-word model, a transcription model and an embedding model, and then deterministic code. I deliberately didn't put a language model in the path — latency, battery, and the fact that it would make the product less predictable, not more."

The second thing it is not: **it is not a search box.** Search asks you to remember what you are looking for. The whole premise is that you cannot — that the work is real and the label for it is not in your head at 9am. If you ever describe Evo as “search over your stuff”, you have explained it as a feature of Spotlight and lost the argument before you started.

The third: **it is not a productivity score.** Nothing in the build rates you, grades your day, or tells you that you were 62% focused. That entire genre of product is a different thesis — it assumes the problem is discipline. Yours assumes the problem is continuity. Say the difference out loud, because it is one of the few places where you can cleanly separate yourself from a crowded shelf.

The sentence that survives interruption

There are four seconds of speech above, and you will rarely get to deliver them in order. An associate will interrupt at second two with a question about whether this is just window management. So what you actually need is one sentence that carries the whole idea and survives being cut off:

> The work already happened. The product reopens it.

Everything else in this section is elaboration. If you get cut off, come back to that sentence, then answer the interruption, then say: “and the reason that’s hard is the two works problem” — which is §07.

THE DEEPEST ANSWER YOU HAVE

03 The evaluation harness, file by file

This is the single most important technical section in either book, because it is Thomas's stated area of interest. He wrote that the field's missing half is owning evaluation . Almost no pre-seed founder arrives with an answer. You do.

First, the distinction that makes you sound like an engineer rather than an enthusiast:

> **SAY THIS**
> The corpus is an answer key, not an input. It never touches the running product. If the engine learned from the data it is graded on, it would be marking its own homework.

What exists, in five files

File	What it is	Size
tools/threads-regression/sample_events.json	One real recorded day from a real machine, exported from the observation log. Each row is [timestamp, origin, interaction, resource, weight].	10,392 events, 7,485 distinct resources, 13–15 September 2026
tools/threads-regression/ground_truth_labels.json	Hand labels: a resource path and the body of work it belongs to.	164 labelled resources across six bodies of work
crates/evo-threads/src/ground_truth.rs	Folds label synonyms so the answer key is internally consistent — several labels name the same work in different words, and the map makes them one.	43 lines
crates/evo-threads/tests/ground_truth.rs	The gate, inside cargo test. Replays the corpus through the real engine and hard-asserts the properties.	207 lines
crates/evo-threads/src/bin/regression.rs	The same gate as a CLI, with a readable report and exit(1) on violation.	226 lines

What the gate actually asserts

Six checks, in order. Learn these; they are the substance behind the word "evaluation."

1. **The corpus is real.** More than 100 events, and `Typed` events must be present. That second one looks pedantic and isn't: without input evidence, you cannot tell whether a resource was worked in or merely looked at, so placement claims become untestable. The test refuses to run on a corpus that can't disambiguate.
2. **The labels are real.** More than 20 hand labels, so a stub can't pass.
3. **Something is reconstructed.** At least one body of work must be minted from a real day. An engine that outputs nothing cannot fail a safety test, so this closes that loophole.
4. **No fusion.** No thread may contain two different labelled works. This is the expensive failure — a client's material appearing in another client's room.
5. **No leisure anchor.** A resource labelled leisure may never become the anchor of a body of work. Leisure can ride along as context; it cannot found a work.
6. **Determinism.** The same corpus, replayed, must produce identical thread identities. If replay is not deterministic, every other guarantee is unfalsifiable.

The CLI version adds the two numbers that should trend: **colocated pairs up, fused threads at**

zero. That is the honest scorecard — not an accuracy percentage.

What it prints, today

> threads minted: 2
> same-work pairs: colocated=1 split=0
> threads fusing 2+ distinct works: 0
> leisure resource ever anchors a thread: false
> OK
>
> — The committed gate's output on the committed corpus

Now the part that makes you credible

Volunteer all four of these before he finds them. They are the difference between "I have an evaluation harness" and "I know exactly what my evaluation harness is worth."

Limitation	Why it matters	How to say it
Two threads from two days	A conservative engine that mints few works passes a no-fusion test almost by construction.	"It mints two works from that day. That's a low number, and it's low partly because the engine is conservative and partly because it's early."
One person, six works	Not a distribution. It tells you nothing about a designer in Berlin.	"It's my own machine. That's the honest sample size and I'm not going to dress it up as a benchmark."
It proves safety, not accuracy	No fusion is necessary, not sufficient. An engine that restores nothing passes.	"It proves I haven't broken the safety properties. It doesn't prove I'm right often enough. That's the next layer."
Colocation is reported, not asserted	The positive direction is a trend line with a value of one.	"The positive metric is reported rather than gated, because with one colocated pair a gate would be theatre."

WHY THIS WORKS

That last observation — that gating a metric with a sample size of one would be theatre — is the kind of thing a research-minded investor respects more than any number. It shows you understand what a test can and cannot prove, which is literally his thesis.

THE ANSWER YOU CANNOT BLUFF

04 Does it learn?

You already know the engine is dumb in this respect. Here is the part that matters: **this is the worst possible investor to bluff on it.** Thomas co-authored DSPy. His published position is that the hard half of AI is problem definition and owning evaluation. If you say "it learns," the next three questions are what's the update rule, what's the label, where's the loss — and there are no answers in the repository.

What is true today

- **Anchors do not decay.** Once you produce in a resource, it stays identity. What you made doesn't stop being what you made.
- **Companions do decay** — a three-day half-life. Context drifts; identity doesn't.
- **Declarations are permanent and outrank inference.** `SameWork` , `SeparateWork` , `NotMine` , `Classified` , `WorkDismissed` , `MemberState` . The code calls them the supreme court of attribution.
- **A merge requires a declaration.** There is no function reachable from inference that can merge two threads. The merge code path takes a declaration or it does not run.
- **Episodes are content-tested.** Resumption is decided by whether you touched the same material, not by whether enough time passed.

So the picture of your work genuinely gets richer with tenure. What it does **not** do is change its own weights.

The sentence to use

> **SAY THIS**
> "Today it compounds memory, not parameters. Every correction is permanent and outranks anything the engine infers. What it doesn't do yet is fit its own weights — that's the next layer."

Then, if he pushes:

> **SAY THIS**
> "Correct. It accumulates evidence today and adapts next. I'd rather show you the boundary than have you find it in diligence."

> **WHY THIS WORKS**
> That answer is worth more than any claim, because it tells him you know which half of your system is which. A founder who can name the exact boundary of his own capability is treated as an operator; one who rounds up is treated as a salesperson.

How it would actually learn — the mechanism

If he asks "how would that work," you need a real answer, not a gesture. There are two loops, and only the second is machine learning.

Loop A — unsupervised, no labels, works today

Count co-occurrence. How often does resource A appear in the same episode as resource B, weighted by production and decayed by time. That is a per-user estimate of P(resource | work) , and it updates continuously with zero user effort. It is statistics, not learning, and it is already how companions and candidate membership behave.

Loop B — supervised, needs corrections

Your declarations are free, high-quality pairwise labels:

What the person does	What it becomes	Label
Merges two works	SameWork { a, b }	positive pair
Keeps two works apart	SeparateWork { a, b }	negative pair
Renames a work	the work's name	a string to score future candidates against
Says "this is media, not work"	Classified	negative class for a surface
Says "not mine"	NotMine	hard negative — permanent
Hides a work from the default surface	WorkDismissed	weak negative on a resume suggestion

The features already exist in the engine: co-focus within a window, shared repository, shared path prefix, same app pair, title-token overlap, embedding similarity, time of day, transition count. The label is 1 or 0. The model is **logistic regression with L2 regularisation** — a few hundred weights, fitted incrementally, on-device, in milliseconds.

The crucial architectural point, and the one to say out loud:

> **SAY THIS**
> "The learned layer doesn't replace the rules. It resolves the near-ties the engine currently forks on. Deterministic core for safety, learned layer for arbitration."

> **WHY THIS WORKS**
> That sentence is genuinely good design, not a fig leaf. It is also the reason you can promise a learner without promising to make the product less predictable — which is the thing a technical investor would otherwise worry about.

Why a tiny model is the right call, not a compromise

He may wonder why you would not just run a small language model over the log. The answer is practical and confident:

- **The decision is narrow.** "Does this belong with that" is a binary call on structured features. Logistic regression is close to optimal for that shape of problem and fully inspectable.
- **It is auditable.** You can print the weights and explain any single decision. You cannot do that with a transformer.
- **It costs nothing.** Microseconds, no battery, no thermal budget.
- **It fails honestly.** A linear model near its decision boundary produces a genuine near-tie — which is exactly the condition where the product already forks and asks.

Cold start and the moat

Cold start: population priors ship inside the binary; the per-user model starts at the prior and moves as evidence arrives. Day one isn't empty; day thirty is better.

Moat, honestly stated: not the model. Anyone can copy a logistic regression. The moat is the per-user evidence graph plus the corrections — six months of this belonged with that , which cannot be copied, only re-earned. Switching cost is tenure, not features. That is the true version of the compounding claim.

The test that would kill the compounding story

If he is doing his job, he is asking himself: how would I know if this is wrong? Give him the answer, because a founder who names the falsifying test is unmistakably serious.

> **SAY THIS**
> "The number that decides whether any of this is real is how often a user corrects the engine. If people correct it a few times in the first week and then stop, the compounding story is dead — it means either it got good enough that corrections don't happen, or people stopped caring. If they keep correcting it for months, the boundary is genuinely hard and genuinely accumulating, and there's a model to fit."

Then the honest caveat:

> **SAY THIS**
> "I don't have that number yet, because nobody is on the current build. It's the first thing I'd instrument after the round, and it's cheap — it's a count of a thing the app already records."

> **WHY THIS WORKS**
> This is a much stronger position than it sounds. Most founders claim a compounding advantage and cannot describe what evidence would disprove it. You have just described the experiment, named the metric, said what result would kill the thesis, and pointed out that the data collection costs nothing. That is research discipline, which is the thing he actually respects.

05 Where does it learn if nothing leaves the device?

This is the sharpest version of the challenge, and it is fair: if the data never leaves, and there is no cloud, how can the product improve? The answer has three parts.

One: the learning is per-user, so it belongs on the device

The model that needs to improve is not a general model. It is your model of your work. There is exactly one user per model. Fitting it anywhere other than your machine would be slower, less private, and no more accurate.

> **SAY THIS**
> "The features are already local — co-focus, shared repository, shared path, title overlap, time of day. The labels are the user's own corrections. Fitting a few hundred weights on a few thousand local pairs is milliseconds on a laptop. The frozen part is the general embedding model; the personalised part is tiny and per-user. There is exactly one user per model, which is why nothing needs to leave the machine."

Two: the analogy that survives a specialist

> **SAY THIS**
> "It's a spam filter. You mark one email as spam and it learns your definition of spam. It doesn't need your inbox on a server, and it doesn't need a frontier model to do it."

Use this one. It is accurate, it is intuitive to a non-specialist, and it does not collapse under technical scrutiny — Bayesian spam filtering really is per-user logistic regression on local data.

Three: what does need the fleet, and what it can safely be

Some things are genuinely better learned across users. Be precise about which, because the answer determines whether the privacy claim survives.

What	Contains personal content?	How it can be shared
App capability map — which apps restore tabs, which restore documents, which restore nothing	No. It is engineering knowledge about software.	Plainly, from day one. No privacy machinery required.
App-pair co-occurrence — how often a code editor and a terminal appear together	Barely. Structural, but still behavioural.	Aggregated only, above a minimum population, never per-user.
Restore-set shapes — how many surfaces a typical work needs	No.	Aggregate.
Content, titles, paths, documents	Yes.	Never. Not aggregated, not anonymised, not ever.

If you do get to the aggregated layer, the vocabulary is settled and you should use the real terms rather than inventing your own:

- **Differential privacy** — calibrated noise on aggregates so no single user can be identified from the result. Apple uses it in production for exactly this shape of problem. Practical budgets sit around ε = 1–10.
- **Secure aggregation** — the server only ever sees the sum, never an individual contribution.
- **Federated learning** — train locally, send updates. **Do not put this on your roadmap.** It is a research programme, it leaks without DP plus secure aggregation, and two people cannot staff it.

What not to claim about privacy

Four sentences that sound strong and are actually traps. Each one invites a follow-up you cannot win.

Don’t say	Why it fails	Say
“Nothing ever leaves your device.”	True today, and it stops being true the moment sync ships. He will ask what happens then.	“Nothing leaves the device today — there’s no HTTP client in the build. Sync is the first time anything would, and it’s designed so the server holds ciphertext.”
“We’re end-to-end encrypted.”	You aren’t. Per-record encryption is roadmap.	“FileVault and the sandbox today. Per-record keys before sync ships.”
“We’re zero-knowledge.”	Zero-knowledge is a specific cryptographic property, and it also implies no metadata leak — which no sync design achieves.	“End-to-end encrypted with minimal metadata: the server sees sizes and timestamps, not content.”
“We don’t collect anything.”	You collect a great deal — it’s just local. The sentence is false and he will notice.	“We collect a lot, and it never moves. That’s the difference.”

WHY THIS WORKS

The last row is the most useful reframe. “We don’t collect anything” invites suspicion because it is obviously untrue of a product that watches your activity. “We collect a lot and it never moves” is both more honest and more impressive, because it names the architecture instead of denying the capability.

CAREFUL

Never say "we'll train on anonymised user data." Anonymisation of behavioural data is a well-known failure, and saying it signals that you haven't read the literature he has. "Aggregated, differentially private structural statistics" is the sentence that means something.

06 Measuring time-to-resume

If he asks what you measure, the honest position is: **the log makes it computable; it is not instrumented yet.** Say exactly that. It converts a gap into a deadline you set yourself.

The three timestamps

Timestamp	Definition	Why this definition
T_interrupt	The last person-origin production event in the episode before it suspends.	Not the first event of the detour — that would measure when you got distracted, which is a different and less useful number.
T_restore	The moment the work is activated, by voice or by click.	The dispatch, not the first window appearing. Windows take time to paint and you don't want paint time in your metric.
T_first_action	The first person-origin Typed, Saved, or corroborated Mutated event on any resource in the restored set, after T_restore.	Action, not focus. A window that opened behind another window was not restored in any meaningful sense.

Time-to-resume = T_first_action − T_restore.

Not interrupt-to-action. That measures how long you were away, which is a fact about your calendar, not about the product.

Two bonuses that fall out of the same query

- **Search-free resumption** — did any navigation to a search engine or file-search surface occur between restore and first action? That is did they have to hunt , which is the thing you are actually selling. A restore that works but sends the user to Spotlight has failed quietly.
- **Restore completeness** — of the restore set, how many opened successfully on the first attempt. That is your fidelity number, and it is already computed: the three buckets are the numerator and denominator.

How it gets computed

A query over the local ledger joined to restore attempts. No server, no SDK, no phone-home. The events carry millisecond timestamps already, so this is a subtraction, not a telemetry pipeline.

> **SAY THIS**
> "Two timestamps and a subtraction, from a log that never leaves the device. Interruption is the last production event before the episode suspends; restore is the moment the work is activated; first action is the first typed or saved event in the restored set. I'm instrumenting it now — I'd rather show you the number in three weeks than invent one today."

> **WHY THIS WORKS**
> The last clause is the whole answer. It converts a missing number into a deadline you chose, which is a display of control rather than an admission of weakness.

The baseline you should compare against

The published figure — 25 minutes 26 seconds, Mark et al., CHI 2005 — is the average time before a worker **returns to the original task** , during which they work on other things. It is not a stopwatch on re-orientation. If you quote it as "time spent reconstructing," a careful reader will catch it.

So quote it correctly, and then say the interesting thing:

> **SAY THIS**
> "That's the return-to-task figure, not a stopwatch on re-orientation. Which is why I want to measure the thing I actually change: seconds from 'continue' to the first productive action. If that's under ten seconds, the product works. If it's ninety, it doesn't, and I'd rather know."

The three numbers you'd publish weekly

Your deck promises to publish retention weekly. That is a good instinct. Here is what a credible public dashboard actually contains — and note that two of the three are not retention at all.

Metric	Definition	Why publish it
Restore completion	Share of restore sets where every target landed in opened or raised.	It is the fidelity number, and it goes down when you ship something that breaks things. Publishing it makes you fix things faster.
Time to first action	Median seconds from activation to the first typed or saved event.	It is the value metric. If this doesn't improve, the product isn't working, regardless of what retention says.
Weekly active restores	Share of activated users who triggered at least one restore in seven days.	This is the actual habit signal — closer to truth than a retention cohort, because it can't be flattered by people who installed and forgot.

Publishing these is a strategic choice, not just a transparency one: it commits you to a number that will embarrass you the week it drops. That is precisely why it is credible.

THE HARDEST QUESTION, ANSWERED

07 Four jobs, fifteen windows

This is the question a technically minded investor will ask, and it is the one that separates you from every competitor: **if someone is working across four projects at once with fifteen apps open, how does the engine know which window belongs to which job?**

The answer, in one line:

> **SAY THIS**
> It doesn't classify windows. It classifies episodes.

The unit is not a window, it's an interval

An **episode** is the smallest thing the stream can honestly recover: a bounded interval where attention stayed on a stable working set. Interleaving is what interrupted people actually do, so boundaries are decided by content-tested resumption first, dip absorption second, and the clock only as a fallback.

Parameter	Default	What it's for
Gap that closes an episode	20 minutes	Lunch, sleep, a meeting. The only boundary that needs no content test.
Detour that gets absorbed	5 minutes	A notification glance is not a change of work.
Person-corroboration window	120 seconds	A mutation counts as yours only if there was focus or typing on that resource just before it. A background service rewriting forty cache files never has a person attached.
Attention credit per event	10 min max	A window left focused overnight contributed one interval, not the night.
Bare focus / bare visit	1 second	Presence with no dwell is honest context, never enough to outrank measured work.
Companion half-life	3 days	Context drifts.
Restore set cap	5	Restoration reopens the neighbourhood, not the museum.

Identity comes from production

Inside an episode, membership is decided by what you made , not by what was open:

- **Anchors** — resources with production: typing, saving, downloading, and mutations that pass the corroboration window. Anchors do not decay.
- **Companions** — attention-only context. Three-day half-life. They decorate; they never vote.
- `CommandRan` alone earns nothing. **A command becomes identity only when a corroborated mutation shows it changed something. Otherwise every shell you glanced at would own your work.**

Assignment, and the three safety valves

When an episode closes, it is matched against live works. The rules, in order of authority:

1. **A declaration overrides everything.** If the person said these belong together, they do.
2. **An uncontested anchor decides** — but only if it dominates both the rival anchor support and the episode's own new production. Old identity must outweigh new identity, or one shared browser drags tomorrow's new work into yesterday's thread.
3. **A shared resource votes for neither.** One browser, one terminal, or one repository serving two real works is safe: its reality is never in question, only its jurisdiction. This single rule is why fifteen windows don't confuse the engine.
4. **A near-tie forks.** Alternatives are retained and shown. It never breaks a tie by guessing.
5. **A merge needs a declaration.** No inference path can merge.

What this means concretely

With fifteen windows across four jobs:

What's open	What the engine does	Why
A browser with tabs from three jobs	Abstains. It becomes a companion or a fork, not an anchor.	It was used for more than one work, so it cannot claim any of them.
A terminal with four repositories	Votes only for the repo that produced something.	The directory you cd'd into but never wrote in is context.
A design file you've been editing	Anchors its work.	Production.
A slide deck you read for reference	Companion. Decays.	Attention only.
A video you had playing	Companion at most. It may never anchor a work.	It is context, not production — and the test suite hard-asserts this.

The honest limitation

> **CAREFUL**
> Genuine second-scale interleaving is hard, and work with no production at all is the thinnest part of the engine. Pure reading — a research session where you only read — is handled by cross-day recurrence rather than by production. If he drills there, agree with him: "Reading-only work is the weakest case and I know it. It's handled by recurrence across days, and it's the next thing I'd test properly."

VOLUNTEER THESE

08 The four things it cannot do

Nothing buys credibility faster than a founder naming his own limits in technical language, in one breath, before being asked. These are the four. Say all four in about twenty seconds and then stop.

Cannot	The precise reason	What it means for the product
Learn in its weights	There is no fitted per-user model. Declarations override; nothing consumes them as training data yet.	It gets better at remembering with tenure, not at deciding. That's the next layer, and it's the honest version of the compounding claim.
Restore a browser session	It can reopen a URL, and there is an adapter that can focus an exact tab in a configured profile — but authenticated sessions, form state and navigation history are not restored.	For most users this is fine, because the login persists in the browser. For a few it's the difference between "back" and "nearly back."
Recover unsaved buffers	If you didn't save, nothing observed it. The only way to claim otherwise is continuous screenshots, which is a different product with a different trust model.	It is also a reason to prefer this architecture: the honest failure is clearly the app's fault, not a silent data loss.
Run anywhere but macOS on Apple silicon	No signing or notarisation pipeline, no auto-update, no Windows.	The repo says so plainly in its own release checklist. Saying it yourself makes everything else you say more believable.

Why volunteering is the optimal move

WHY THIS WORKS

An associate's prior is that a pre-seed founder is rounding up. When you pre-emptively correct yourself, you invert the prior: everything you don't correct is now presumed accurate. You are buying credibility with information he was going to discover anyway — the cheapest trade available in the meeting.

CAREFUL

The mistake is overcorrecting into self-flagellation. Four limits, twenty seconds, once, calmly — then forward. Five minutes of listing weaknesses makes him conclude that you don't believe in the company.

09 Privacy is the licence, not the moat

There is a sentence in your own material that is almost right, and fixing it is worth real money in the meeting. The version that survives scrutiny is this:

> **SAY THIS**
> "Privacy is not the moat; it's the licence to operate. The moat is that the classification model is personal: it learns your bodies of work from your evidence, on your machine. That's not a dataset I can pool, and it's not one a competitor can copy — they'd have to start from zero with the same user."

Grade the sentence, clause by clause

Claim	True?	Strength
"Privacy is the licence to operate, not the moat"	Yes	Strong. Say it exactly like that.
"Per-user compounding state is the moat"	Yes — and the architecture already produces it	The strongest thing you own.
"Fleet-level structural priors are part of the moat"	Half true	Weaken this. It is a quality lever, not a defence.

Why fleet priors are not the moat

Think about who wins a priors race: whoever has the most users. That is a scale game, and you would be the smallest player in it. If priors were the moat, the company with 66,000 GitHub stars and an AWS partnership would already own it.

Prize the priors for what they actually are — **cold start** . They make user 1,001 feel as smart as user 1. That is a retention and onboarding advantage. It is genuinely valuable. It is not defensible.

What a competitor cannot copy is that your user has six months of declarations they would have to re-earn somewhere else. That is tenure, and it is the only asset in this company that gets harder to replicate every day.

> **SAY THIS**
> "The state that compounds is per-user — which windows belong to which body of work — and it can't be pooled, so a competitor starts from zero with the same person. That's my moat. Privacy is what lets me hold it at all."

JURISDICTION

10 The category is bigger than memory

Your instinct that this could be enormous is right, but the framing matters. "Personal agents trained on your data" is the weak framing, because it invites a training question you cannot answer. The stronger framing is narrower and much better:

> **SAY THIS**
> "Every agent has a memory problem. Almost none have a jurisdiction problem — and jurisdiction is the one that causes catastrophes."

The distinction, concretely

- An agent with **memory** knows you worked on three things this week.
- An agent with **jurisdiction** knows that this repository is client A, that deck is the pre-seed raise, and that browser tab belongs to neither. So it knows what it must not touch, must not send, must not merge.

Every personal agent shipping today — ChatGPT, Claude, Cursor, Copilot — fails the second test. They will happily write your side-project code into a client repository because nothing told them where the boundary was. Memory products store facts about you . None of them store boundaries between your works .

Why the gap exists

Boundaries require production evidence. You can infer "this person likes TypeScript" from a chat log. You cannot infer "these forty windows are one job and those twelve are another" from text — you need to see what was created, where, and in what order. That means being resident on the machine, continuously, for months.

That is a product problem, not a model problem. Which is exactly why a lab with a trillion tokens still cannot ship it.

The five sentences

1. Privacy is the licence to operate; the moat is the per-user state that compounds with tenure.
2. Fleet priors are cold start, not defence — I'd say that plainly rather than oversell them.
3. The gap isn't memory, it's jurisdiction: no agent knows where one body of work ends.
4. I don't need a learner to be the jurisdiction layer — I need a queryable boundary, and that exists.
5. Every agent gets memory. Almost none get boundaries. That's the company.

KNOW THEM COLD

11 Who else is building this

"Nobody else is doing this" is a claim that gets checked in ten minutes. Here is the landscape, and the one-line read that puts you in a different category.

Player	What they actually do	Why it isn't you
Mem0	A "memory passport" — memory that travels across apps and agents. $24M raised across seed and Series A, and the memory provider for AWS's Agent SDK.	Stores extracted facts, cloud-first. No concept of work boundaries.
Letta (ex-MemGPT)	Memory the agent owns and edits itself. $10M seed at a reported $70M post.	A runtime for agents, not a model of your desktop.
Zep / Graphiti	A temporal knowledge graph that tracks how facts change over time.	Tracks facts. Not jurisdiction.
Supermemory	A personal memory API. Small seed round.	Developer API, not device-resident.
Glean	Enterprise context graph plus agents. $7.2B valuation, roughly $300M ARR. Already ships a personal graph per employee tracking projects and work patterns.	Enterprise only, indexes documents. And its existence proves the category, at the top of the market rather than the bottom.
Rewind / Limitless	Screen-recorded personal memory. Acquired by Meta in December 2025; the Mac app was discontinued.	Recorded pixels. Retrieval only, no restoration.
Microsoft Recall	On-device screen snapshots plus a semantic index. Shipping, opt-in, Copilot+ PCs only.	Windows only. Retrieval only.
OpenAI Computer History	Interaction events from permitted apps, processed server-side into memories, stored locally as Markdown. macOS, opt-in, Pro/Business/Enterprise.	Makes ChatGPT remember. Doesn't restore your workspace, and the event stream leaves the machine to be processed.

The one-line read

SAY THIS

"Every one of them answers 'what do I know about this person?' None of them answers 'which body of work does this belong to, and is it safe to act here?'"

The phrase to avoid

CAREFUL

Mem0's pitch is that your AI memory travels with you across apps, like email or logins. If you say "your work context travels with you across agents," you sound like a worse-funded Mem0. Say boundary, not portability.

FOUR PHASES

12 How the layer actually gets built

A vision without a sequence is a liability. This is the sequence, with a gate before each phase. Do not skip ahead — the most common way this vision dies is by building the platform before the wedge works.

Phase	When	What you build	The gate
0. The asset	Now	Nothing new. Make the declaration log a first-class, exportable artefact — every merge, split, rename, classify.	You can show a stranger six months of one user's corrections.
1. The capability map	Month 2–6	Which apps can restore what: full, partial, or none, per app family.	~50 app families documented. And it contains no personal data at all.
2. The local API	Month 6–12	A read-only query over the existing daemon socket: what is the current work, what's in it, what was I doing.	One external agent consumes it successfully.
3. Fleet priors	Seed, 5,000+ users	Content-free aggregates only: app-pair co-occurrence, transitions, restore-set shapes.	1,000+ users per bucket, or the prior is noise.
4. DP aggregates	Series A	Differentially private aggregates with a published ε budget.	Legal review and a privacy engineer on staff.

Phase 1 is the sleeper

The capability map is not personal data. Knowing that one app restores tabs but not canvas state, that a second needs working-directory capture, that a third restores poorly — that is engineering knowledge. It is the same for every user on earth.

It costs you nothing in privacy, it improves restoration for everyone, and **it is the thing a competitor would have to re-derive app by app.** Accumulate it from day one, in a structured file, and you will have a real asset by month six that nobody can copy from your website.

That is your first genuine fleet-level prior, and it requires zero differential privacy, zero legal review, and zero users.

What already exists toward this

- **The declaration log** is in `evo-threads/src/types.rs` and it persists. It needs an export, not a redesign.
- **The socket** already exists — `classification.rs` binds a Unix listener at a hashed path. A read-only query is a small addition.
- **An MCP server on top of that socket** would be the distribution mechanism for the whole thesis: any agent that speaks MCP can ask Evo what this person is working on. Local, no cloud, no key management. That is a weekend of work, and it makes the vision demonstrable instead of rhetorical.

> **NOTE**
> Notice what that sequence implies: the platform story becomes real before the model story does. You don't need a learner to be the jurisdiction layer. You need a queryable boundary. You already have one.

THE ANSWER THAT DECIDES IT

13 "Is it big enough?"

This is the question you are most likely to lose, because the instinct is to answer it with a market-size number. Don't. He has heard "$1B knowledge workers" four times this week.

The answer, in three moves

SAY THIS

"As a paid Mac utility, honestly, no — that's a five to forty million dollar ARR business and I'd be fine saying so.

The reason I think it's bigger is that the thing we accumulate isn't a feature, it's the boundary map of a person's work. Every agent shipping right now has memory and no jurisdiction — it knows facts about you, it doesn't know that this repository is one client and that deck is a different job. So it can't safely act on your desktop.

We produce that boundary as a side effect of being useful every morning. Then agents query it. That's the layer."

Then stop. Do not add a TAM number. The concession is what makes the larger claim credible, and adding a number after it undoes the effect.

Why conceding works

WHY THIS WORKS

A founder who says "the wedge alone is a $5–40M business" has just demonstrated that he can size his own market honestly. That makes the second claim — that there's a layer above it — land as analysis rather than fantasy. Founders who will not concede the small version are not believed about the large one.

If he pushes on the arithmetic

He asks	You say
"Where does $5–40M come from?"	"A paid Mac utility at $108 a year, with realistic conversion on a professional population, tops out somewhere in the low tens of millions of ARR. I'm not going to pretend otherwise — the interesting question is what happens when agents need the boundary."
"Why would an agent pay for this?"	"Because acting on someone's machine without knowing where one job ends is the failure mode that stops agents being deployed at all. The boundary is a safety primitive, not a convenience."
"Can't OpenAI just do it?"	"They can do it for ChatGPT. They can't do it for every agent, on every machine, and they can't do it without the events leaving the device. Neutrality is the product."

14 The four answers, rewritten

Four answers in the earlier draft were weaker than they needed to be. Here they are again, in the form that survives a follow-up. These are the ones to rehearse.

T2 — "How do you evaluate classification?"

SAY THIS

"Two things, and I'll separate what exists from what's next.

What exists: a committed evaluation corpus from a real recorded day of mine — about ten thousand events, a hundred and sixty-four hand-labelled resources across six bodies of work. It replays through the engine inside the test suite and hard-fails on three properties: no thread may fuse two labelled works, no leisure resource may anchor a work, and replay must be deterministic. The same check runs as a CLI and exits non-zero.

What that does and doesn't prove: it proves I haven't broken the safety properties. It does not prove accuracy — it's one person, two days, and a conservative engine mints few threads, so a clean run is necessary and not sufficient. I know that.

Next is a shadow replay — real logs through today's engine compared to the last accepted output — so drift shows up on data I never labelled. After that, per-user evaluation against the user's own corrections, which is the only ground truth that scales."

T3 — "What happens when it gets wrong?"

The earlier draft led with "it will get wrong," which sounds like an apology. The better answer is structural:

SAY THIS

"It gets things wrong. The design question is what happens next, and the failures aren't symmetric.

Fusing two works is expensive — a client's material shows up in another client's room. Splitting one work is cheap — you merge it back in one gesture. So the engine is built to prefer splitting. When two candidates are close it forks and shows you both instead of guessing. A resource shared by two works votes for neither. A merge only happens if the person declares it — there is no code path from inference to a merge.

Restoration is capped and reversible: every activation produces a lease that replays in reverse. And the surfacing rule is under-containment — a window I'm not sure about stays visible rather than being hidden.

So wrongness is visible, cheap, reversible, and one gesture from permanent. The failure I design against is the silent merge, and the system cannot do one."

T9 — "Storage and performance at 100,000 users?"

Add the two things missing from the earlier draft: the reason, and a number.

SAY THIS

"Per user, the ledger is event metadata — kilobytes to low megabytes a day depending on activity — and the index is incremental. At a hundred thousand users the infrastructure cost is the sync service and the entitlement check, not the observation, because observation never leaves the device.

Concretely: adding user ten-thousand-and-one costs an entitlement check, some update bandwidth, and support. Call it cents to a quarter a month against nine dollars of revenue. That's why gross margin is near the ceiling — there's no inference bill. The intelligence runs on the user's laptop, not on my invoice."

T11 — "Encrypted sync without you being able to read it?"

SAY THIS

"The design constraint is that the server stores bytes it cannot read.

An account master key is derived from a passphrase, or generated with a recovery code. Each device gets its own keypair, its private key wrapped by the master key. Each synced object gets a random data key wrapped by the device key. The server holds wrapped keys it can't unwrap. Enrolling a new device needs an existing device or the recovery code.

The server does see routing metadata — account, device, object ids, sizes, timestamps. So it's end-to-end encrypted with minimal metadata, not zero knowledge, and I'd rather say that precisely than use a marketing term.

Conflict resolution is why the append-only ledger matters: two devices that saw different things both contribute evidence, and the projection is a pure function of the event set, so they converge. Deletions are tombstones. Revoking a device rotates the key — the device keeps what it already downloaded, which is the honest limit of any scheme like this.

I haven't built it. Month two to five, and it's why the round is 1.5 and not 500K."

CAREFUL

Do not claim CRDT expertise, "military-grade" anything, or zero-knowledge. The append-only plus deterministic-projection argument is true and sufficient. Inventing a sync architecture on the call is how founders get caught.

15 The demo when the model isn't learning

You said the engine isn't learning from new data yet. That is fine — **do not demo learning.** Demo memory and correction. Those are real, visible, and they are the product people actually fall in love with.

Thirty minutes before

1. Confirm the daemon has events for today. A cold machine demos nothing.
2. **Create production in each of the three works you plan to show.** Type a line, save a file, run a command, commit something. Production is what mints identity — a work you only looked at may not exist as a work.
3. Open the pod bar and confirm three works. **If one is wrong, fix it now with a merge** — and then that becomes a demo beat instead of a liability.
4. Have the recording open in a tab, and say so before you share the screen.

The four beats

Beat	What you do	What it proves
Resume	"I was on the pitch deck Friday." Then: "Hey Evo, continue the pitch deck." Let it rebuild. Point at the clock.	Restoration is an action, not a search result.
Memory across time	"It knows this from a session two days ago. I didn't tell it."	State is durable rather than session-scoped.
Separation	Two works that share a browser. "Same browser, both jobs. It doesn't guess which — it abstains and asks."	The hardest technical claim, made visible. Nobody else can demonstrate it.
Correction	Drag a window into the wrong pod. It merges and stays. "One gesture. Permanent."	Wrongness is cheap, reversible, and outranked by the person.

The video tab — a warning

CAREFUL

Your own gate hard-asserts that a leisure resource never anchors a work. If you play a random video and Evo files it into a work, you are demonstrating the exact failure your test suite forbids. Only include it if it is genuinely reference material for that work — and if it is, frame it as such: "this tab is context; notice it can't found a work. Only production does that." That is a superb beat, but only if it's true.

If he asks "is it learning right now?"

SAY THIS

"Not in the weights. It's accumulating evidence — that's why the pod has more in it today than it did on Tuesday."

16 Incorporation, visas, and the $83,000

The one-line answer

SAY THIS

"We're incorporated in Delaware — that's the entity you'd wire into. Neither of us needs a visa to run it or to raise. What we need a visa for is being on the ground, and the round funds that: we file I-941 under the International Entrepreneur Rule on close, because the investment clears the threshold by several times. Worst case we're on O-1s. Either way it doesn't gate the business."

The facts behind it

Route	What it requires	Your position
International Entrepreneur Parole (I-941)	A US entity formed within five years; at least 10% ownership; a central and active role; and $311,071 in qualified investment from a qualified investor within the prior 18 months. Grants up to 2.5 years, renewable once. Up to three founders per company.	A $1.5M round clears the threshold roughly five times. NEA clears the qualified-investor test without effort. Filing costs around $2,300 per person in government fees.
O-1A	Three of eight criteria. Since January 2025 a founder's own company can petition for them, provided there is an independent director with authority. No cap, no lottery, three years initially, premium processing in fifteen business days.	The fallback. Genuine technical contribution, expert letters, and press are the three most plausible criteria.
B-1	Business visitor: meetings, demos, conferences, negotiating contracts. Not productive work.	What you use in the meantime, including if he asks you to visit the office.
H-1B	Lottery, 85,000 cap, and a fee regime in and out of court.	Ignore it. It is not a founder path and mentioning it sounds like you googled "US visa" ten minutes ago.

The trap

CAREFUL

Do not tell him you are moving to San Francisco on a date. If your pitch is "we relocate after the round," you have told an investor that your go-to-market depends on a discretionary government decision you do not control. Parole is not a visa: it is granted case by case, it cannot be appealed if denied, and it does not lead to a green card on its own.

Say instead: **the US is the market before it is the address.** Two founders in San Francisco on $83K a month is a choice; two founders in India with a US sales presence is a strategy — and it is why $1.5M buys eighteen months instead of six.

The budget

He will divide 1.5 by 18 in his head and get $83K. An associate who thinks your burn was derived from your raise rather than the other way round will probe. So present it bottom-up:

Line	Monthly	Note
Two founder salaries	$16.7K	2 × $100K a year. Modest, and say so.
Two engineers from month 4	$25.0K	The first hire is the senior Rust / macOS engineer.
Payroll tax and benefits	$7.5K	About 18%.
Co-working	$2.5K	Cheap. Do not claim an SF office.
Legal, accounting, immigration counsel	$2.5K	
Infrastructure and dev accounts	$1.3K	Low, because there is no inference bill.
Recruiting (amortised)	$3.0K	A senior systems hire costs real money to source.
Travel and events	$3.0K	US customer and investor travel.
Contract design and security review	$5.0K	
Insurance and buffer	$4.0K	Don't skip this line; it signals maturity.
Base	~$70K	21 months.
With buffer and FX	~$83K	18 months.

The number you cannot get wrong

$12M is not aggressive. Carta's Q2 2026 data puts the median post-money SAFE cap at about **$18M** for rounds of $1M–$2.4M. You are below median. Know that before he tells you, because a founder who thinks he is being expensive when he is actually cheap negotiates badly.

The same logic applies to the amount. **$1.5M on a $12M post is 12.5% dilution** — squarely inside the normal band, and slightly on the founder-friendly side. If he pushes the cap down, the thing to protect is not the number, it is the option pool and the information rights. A cap that moves by a million costs you about one point; a 15% pool costs you three.

If he says	What it means	What you say
“The cap feels high.”	Standard opening. He is testing whether you have comparables or a feeling.	“Median for this size is eighteen. I’m at twelve. I’d rather be cheap and fast than expensive and slow.”
“We’d want a bigger round.”	NEA prefers to own more. Not hostile — a preference.	“Then let’s talk about what the extra two million buys and whether it changes the eighteen-month plan.”
“Can you get a lead?”	He is asking whether he has to do the pricing work.	“I’m talking to three others. I’d rather NEA set the terms.”
“We’d like a board seat.”	For $1.5M at pre-seed this is a real ask and usually not warranted.	“Board observer now, seat at the next round — and I’ll say why: at this stage a board is one more meeting, not one more advantage.”

THE MOST USEFUL SKILL IN THE ROOM

17 When you don't know

He can ask anything and you cannot know everything. What he is testing is not your knowledge — it is whether you **know the edges of your own knowledge** , because that predicts whether you will be honest with him for the next five years.

The four beats

1. **Acknowledge.** "I don't know — that's a real gap." No flinch, no filler.
2. **Nearest true thing.** "What I do know is ___."
3. **How you'd find out, and when.** "The way I'd answer it is ___; I can have that to you by Tuesday."
4. **Turn it.** "How would you test it?" or "What would you need to see?"

> **SAY THIS**
> "I don't know. That's a gap, not a hedge. What I do know is the shape of the answer — it's a storage-growth question and I'd measure it on my own machine over a week rather than guess. I can send you the number by Wednesday. How would you sanity-check it?"

For design questions — show the axes

If it is not a fact you're missing but a decision you haven't made, do not fake an answer. Show the trade-off:

> **SAY THIS**
> "That's a decision I haven't made. The trade-off is privacy against usefulness: cross-device sync is the thing people ask for, and it's also the first time anything leaves the machine. My instinct is encrypted sync where the key never touches the server, but I haven't pressure-tested it. How have you seen local-first companies handle that?"

The three you genuinely don't know

Question	Answer
"What's your storage growth per user per day?"	"Order of megabytes — the ledger is event metadata, not content. I haven't measured it precisely and I'd rather instrument it during the beta than give you a number I invented."
"What does Windows cost you?"	"Scoped, not built. Capture is weeks; restoration is months per app family. I haven't written a line of Windows code."
"What's your CAC?"	"Unknown. I haven't spent a dollar on acquisition. My hypothesis is that the demo clip carries the launch and the first cohort is organic — and the launch budget exists precisely to find out."

What never to do

- **Never invent a number.** One fabricated figure, discovered later, retroactively invalidates everything true you said.
- **Never say "I'll get back to you" without a date.** A promise without a deadline is a deflection, and he will notice.

- **Never argue with a question.** If he says "this looks like a feature," do not say "no it isn't." Say "that's the risk I think about most" and then give him the counter-argument.
- **Never blame the market, the timing, or other investors.** Fastest way to look junior.

The recovery, in one paragraph

When an answer goes badly — and one will — do not try to rescue it in the same breath. The recovery has three beats and takes about fifteen seconds:

1. **Name it.** “That was a bad answer.” Said plainly, without a laugh, it buys you more credibility than a good answer would have.
2. **Say what the good answer was.** One sentence. The version you would have given if you had thought for another ten seconds.
3. **Move.** Ask him a question, or return to the thread. Do not apologise twice and do not explain why you were nervous.

The reason this works is that he is not scoring your performance. He is trying to find out whether you are the kind of person who can be told something is wrong in month four and act on it. A founder who names a bad answer cleanly has just shown him that, in fifteen seconds, with no preparation. That is worth more than a polished answer to a question he has asked forty times.

What not to do in that moment: do not fill the silence with a longer version of the bad answer. Volume reads as panic, and panic is the one thing in the room that is actually contagious.

Six answers you will not have on Friday

Read this the night before. These are the questions where the honest answer is short, and where a short honest answer is stronger than anything you could rehearse.

If he asks	The honest one-liner
“What’s your retention?”	“Unknown. Nobody is on the current build. The previous build showed day-30 retention I won’t quote, because the architecture changed.”
“What’s your CAC?”	“Zero, because I’ve spent nothing. When I do spend, I’ll know the number in a week.”
“Who else is doing this?”	“Mem0, Letta and Zep on memory; Glean on work. None of them separate two works that share a screen.”
“Why won’t Apple do this?”	“Apple indexes files, not work. Recall ships on-device and still can’t tell you which of two projects a PDF belongs to.”
“Why are you the one?”	“I have the pain, I’ve had three hundred conversations about it, and I’ve already built the gate that fails when I’m wrong.”
“What happens if this doesn’t work?”	“The engine is a local activity-and-artefact model with a verifier. That generalises to any desktop agent that has to know what a person is doing.”

IF YOU KEEP NOTHING ELSE

18 The five sentences

These are the five that carry the most weight per word. Learn them as ideas rather than as lines, so they survive being paraphrased under pressure.

1. **"It compounds memory, not parameters — and the learned layer is next, fitted on-device from your own corrections."**
2. **"The corpus is an answer key, not an input; it's six works from one day, and I know exactly how small that is."**
3. **"Failure is designed to be cheap: fork instead of guess, abstain on shared resources, never merge without a declaration."**
4. **"Time-to-resume is computable from a log that never leaves the machine, and I'm instrumenting it now rather than quoting it."**
5. **"Marginal cost is near zero because there is no inference bill — the intelligence runs on the user's laptop."**

One last thing, and it is the thing to remember when your hands are shaking at 11:29pm.

> You got this meeting because you wrote one specific email to one specific person about something she had actually written. That is the same skill this call requires: say the true, specific thing to the person in front of you, and then stop talking. You already know how to do it.

Everything else in this book is preparation for the moment when you have to do that for thirty minutes straight.